

Audion Image Tools

Windows 10 | 11

release no releases or repo not found

downloads repo not found

license repo not found

Version 2.4.1 · 2026-08-24 · 298.0 MB

- [Direct download](#) — unmetered, no rate limits
- [Project page](#) — every version and how to install

SHA-256: 200bfe689c13f5d85a571a7a391465d89a49a0ce0752dd40dc784374ae8592e3

Portable image conversion and transformation toolkit based on the Audion Python Portable Template.

The project follows one practical rule:

- accept and convert as many real-world input formats as possible
- keep the deepest, safest, and most predictable workflows centered on JPG and PNG

`launcher_tools.cmd` stays untouched from the template. The main daily entry point is `launcher_gui.cmd`. `launcher_project.cmd` and `launcher_project_ru.cmd` remain available for CLI/TUI workflows. Both project launchers work in dual mode:

- the main menu uses FZF when it is available
- if FZF is not available, the launcher falls back to plain CMD menus
- critical short nested choices use plain `Select` or manual input to avoid value mix-ups on deeper steps

What the project does

- convert supported input to JPG/PNG/TIFF/WebP/AVIF/HEIF/HEIC
- main conversion quality: quick presets 60/75/90 plus an exact 1..100 spinner, defaulting to 83

- PNG is exposed in the GUI as a simple quick action; CLI can still use explicit compression options when needed
- WebP / AVIF / HEIF / HEIC output for compact delivery while keeping JPG as the compatibility fallback
- EXIF rotation fix
- DPI update without resampling
- color normalization with explicit profile choice
- sRGB choice: Pillow `Create Profile sRGB` or `sRGB IEC 61966-2.1` from color.org
- CMYK choice: `Photoshop5DefaultCMYK.icc` or `CoatedFOGRA39.icc`
- safe alpha handling for JPG output
- grayscale conversion
- screen resize to 1080p / 1440p / 2160p
- ratio fitting for 16:9, A4, and A3
- roll fitting: pixels stay unchanged, DPI is calculated from the selected side in millimeters
- DPI-only mode: pixels stay unchanged, the selected DPI is written as metadata
- solid border trim
- smart white-background crop
- optional crop safety margin in millimeters
- PDF build immediately after crop with `PNG embedded` or `JPG(75) embedded`
- watermark text overlay: horizontal corner label or diagonal protective text at about 60% of the image diagonal, with adjustable opacity and color
- tile A5/A4/A3 sheets or a custom-width roll with one image for sticker work
- inspection contact sheet generation with filename, a `px | cm | DPI` size line, KB labels, explicit `List` / `Custom` size mode, common thumbnail-size dropdown, selectable output format, and 75/92 quality choices
- split multi-page TIFF into numbered PNG frames
- split images into strips or rectangular grid cells, up to 100 parts, with PNG as the default and selectable output format
- downscale to 25%, 50%, a custom percent, or automatic fit to a target file size in MB
- resize algorithm choice: `Lanczos`, `Bicubic`, `Box`, `Nearest`; `Box` and `Nearest` are especially useful for PNG maps, plans, and flat graphics
- simple PDF export to JPG/PNG/TIFF/WebP/AVIF/HEIF/HEIC with `embedded` or full-page `render` mode

- `pdf-export --mode render` accepts arbitrary DPI from `1` to `1200`
- PDF page rasterization to JPG/PNG/TIFF/WebP/AVIF/HEIF/HEIC at 150 / 300 / 600 DPI
- PDF embedded image extraction as original files or converted to JPG/PNG/TIFF/WebP/AVIF/HEIF/HEIC
- images to PDF in lossless-preferred mode or JPG High 75 / 90
- PDF normalization bridge: PDF -> TIFF pages -> normalize -> PDF

Color workflows

This is the part that matters most if you are deciding how to use the tool.

sRGB output

Use this when the target is screen viewing, web delivery, or general-purpose RGB normalization.

The project offers two explicit sRGB paths:

- `Create Profile sRGB (Pillow)`
- `sRGB IEC 61966-2.1 (color.org)`

CMYK output

Use this when the target is print-oriented output and you need a CMYK conversion instead of a generic RGB result.

The project currently offers two explicit CMYK profiles:

- `Photoshop5DefaultCMYK.icc`
- `CoatedFOGRA39.icc`

Transform vs embed

The project treats two different ICC tasks separately:

1. use an ICC profile as the math for color conversion
2. embed an ICC profile into the output file as metadata

This distinction matters for both print and PDF workflows.

For this project, the important default is:

- `sRGB` normalization converts pixels and embeds the resulting sRGB profile by default

- **CMYK** normalization converts pixels through the selected CMYK ICC profile, but does not embed that CMYK profile into each output image by default

Why this policy exists:

- color conversion is the part that changes the actual image data
- embedded ICC is only a description attached to the already-converted pixels
- large CMYK ICC profiles can massively bloat batches, Office files, and PDF image parts if they are attached to every output image

For print-oriented CMYK work, the practical success criterion is not "does every image carry a CMYK ICC blob", but "do the converted pixels print correctly in the intended workflow".

PDF and print policy

PDF normalization follows the same transform-versus-embed logic.

- PDF pages are rasterized to TIFF
- the TIFF pages are normalized through the selected sRGB or CMYK profile
- a new PDF is assembled from those normalized pages

By default, the project keeps profile preservation for images that are assembled into PDF, because this can matter in print-oriented downstream workflows.

At the same time:

- CMYK profile embedding is still treated as optional, not mandatory
- the default policy prefers correct CMYK conversion without forcing a heavy ICC blob into every output image
- if a specific print pipeline truly requires embedded ICC in PDF image parts, that behavior can be enabled separately from the transform itself

PDF normalization

PDF normalization is page-based, not metadata-only.

The internal path is:

1. rasterize PDF pages to TIFF
2. normalize the TIFF pages with the selected sRGB or CMYK path
3. assemble a new PDF from the normalized pages

Format strategy

Stable core

- BMP
- GIF
- JPG / JPEG
- PNG
- TGA
- TIFF / TIF
- WebP
- AVIF
- PSD as best-effort through Pillow

Extended adapters

- HEIC / HEIF via `pillow-heif`
- CR2 / CR3 / DNG via `rawpy`

Best-effort only

- WMF
- IFF
- XIF

Some formats require optional adapters or platform-specific support. Use the launcher menu or run:

```
python system_core\main.py formats
```

Main folders

- `input\` - quick source folder
- `output\` - processed results
- `logs\` - JSON run reports
- `config\` - defaults, GUI themes, and ICC paths
- `system_core\` - Python implementation

- `install\` - portable runtime and helper scripts
- `wheelhouse\` - offline wheels
- `release\` - release archives

Launchers

Start the GUI with:

```
launcher_gui.cmd
```

English project launcher:

```
launcher_project.cmd
```

Russian launcher:

```
launcher_project_ru.cmd
```

The launcher first tries embedded `fzf.exe`, then `fzf` from `PATH`, then falls back to CMD.

The current launcher workflow is intentionally simplified for daily use:

- the launcher reads source files from `input\`
- the launcher writes results into standard subfolders inside `output\`
- there are explicit `Open INPUT` and `Open OUTPUT` menu entries for Explorer-driven work
- PNG in the GUI is exposed as a simple quick action without compression-preset noise
- PDF export offers `embedded`, quick `render (300 DPI)`, and `render` with a custom DPI prompt

GUI themes are selected in the GUI header. The selected theme is stored in

`config\gui_settings.yaml`; palettes and CSS tokens are stored in `config\ui_colors.yaml`.

The GUI left navigation contains ROOT modules: `Conversion`, `PDF`, `Crop`, `Resize`, `Color`, `Photo sheet`, `Sheets and labels`, `Contact sheet`, and `Diagnostics`. `Conversion` opens directly as a working window: quick actions are arranged at the top, and normal conversion settings are shown below. Parameterized commands show grouped settings and an amber `RUN` button. Maintenance actions are kept in a separate Maintenance block and dangerous actions require confirmation. The global multithreading default also lives there: on `16 GB RAM` keep it off or use `1-`

2 workers, 32 GB is usually comfortable at 4, and 64 GB+ can try 8-12 after checking peak memory.

Short GUI labels are explained through tooltips on headings, buttons, checkboxes, radio controls, and segmented toggles. BACK has no tooltip.

Normalization in the launcher is intentionally simplified:

- Normalize to sRGB, PDF stays PDF asks which sRGB path to use
- Normalize to CMYK, PDF stays PDF asks which CMYK profile to use
- the older broad "normalization with parameter selection" launcher entry has been removed to reduce ambiguity

This means the launcher now makes the profile decision visible at the moment you choose sRGB or CMYK, instead of hiding it under a more abstract advanced menu.

CLI examples

Convert a folder to JPG at the default 83 quality:

```
python system_core\main.py convert --input input --output output\jpg_83 --to jpg
```

Convert a folder to PNG with balanced compression:

```
python system_core\main.py convert --input input --output output\png_balanced --to png --png-preset balanced
```

Normalize to sRGB JPG with white alpha background:

```
python system_core\main.py normalize --input input --output output\normalized_jpg --to jpg --target-profile srgb --srgb-profile-mode colororg --srgb-profile config/icc/sRGB2014.icc --alpha-bg white --metadata preserve_dpi_color
```

Normalize to sRGB JPG using Pillow's built-in sRGB profile:

```
python system_core\main.py normalize --input input --output output\normalized_jpg_pillow --to jpg --target-profile srgb --srgb-profile-mode pillow --alpha-bg white --metadata preserve_dpi_color
```

Normalize to CMYK TIFF using Photoshop 5 Default CMYK:

```
python system_core\main.py normalize --input input --output
output\normalized_cmyk_ps5 --to tiff --target-profile cmyk --cmyk-profile
config/icc/Photoshop5DefaultCMYK.icc --alpha-bg white --metadata preserve_dpi_color
```

Normalize to CMYK TIFF using Coated FOGRA39:

```
python system_core\main.py normalize --input input --output
output\normalized_cmyk_fogra39 --to tiff --target-profile cmyk --cmyk-profile
config/icc/CoatedFOGRA39.icc --alpha-bg white --metadata preserve_dpi_color
```

Update DPI only to 300 without resampling:

```
python system_core\main.py normalize --input input --output output\dpi_300 --to png -
-target-profile keep --metadata preserve_dpi_color --set-dpi 300
```

Resize to 2160p crop:

```
python system_core\main.py resize-screen --input input --output
output\screen_2160_crop --to jpg --target 2160p --mode crop
```

Downscale PNG maps to 50% without creating new colors:

```
python system_core\main.py downscale-percent --input input --output
output\downscale_png_nearest_50 --to png --scale-mode percent-50 --algorithm nearest
```

Downscale maps by a custom percent using dry **Box** averaging:

```
python system_core\main.py downscale-percent --input input --output
output\downscale_png_box_custom --to png --scale-mode custom --custom-percent 37.5 --
algorithm box
```

Fit JPG outputs to about 1 MB per file:

```
python system_core\main.py downscale-percent --input input --output
output\downscale_jpg_1mb --to jpg --scale-mode target-mb --target-mb 1.0 --algorithm
bicubic --jpeg-quality 75 --workers 8
```

Add a diagonal protective watermark at about 60% of the image diagonal:

```
python system_core\main.py watermark --input input --output output\watermark_diagonal
--to png --text "DRAFT" --watermark-mode diagonal --diagonal-coverage 60 --opacity 96
--color "#cc0000"
```

Fit to roll by short side to 900 mm without pixel resampling:

```
python system_core\main.py plotter-size --input input --output
output\plotter_roll_fit --to png --side short --target-mm 900
```

DPI-only without resampling:

```
python system_core\main.py plotter-dpi-only --input input --output
output\plotter_dpi_only --to png --dpi 300 --metadata preserve_dpi_color
```

Split images into 4 vertical strips:

```
python system_core\main.py split-half --input input --output output\split_png --mode
strips --orientation vertical --parts 4 --metadata preserve_dpi_color
```

Split images into a 2 by 3 grid:

```
python system_core\main.py split-half --input input --output output\split_png --mode
grid --rows 2 --columns 3 --metadata preserve_dpi_color
```

Split TIFF into numbered PNG frames:

```
python system_core\main.py split-tiff --input input --output output\tiff_split
```

Tile A4 with 40x30 mm stickers, 7 mm margins, and a 0.2 mm frame:

```
python system_core\main.py tile-sheet --input input --output output\tile_sheet --
paper a4 --orientation portrait --margin-mm 7 --gap-mm 0 --dpi 300 --size-mode doc-
40x30 --frame-mm 0.2 --count-mode fill
```

Build a 600 mm roll with exactly 3 rows:

```
python system_core\main.py tile-sheet --input input --output output\tile_roll --paper
roll --roll-width-mm 600 --margin-mm 7 --gap-mm 0 --dpi 300 --size-mode width --item-
width-mm 40 --count-mode rows --rows 3
```

Crop white background exactly to meaningful pixels:

```
python system_core\main.py smart-crop-white --input input --output
output\smart_crop_white --to png --tolerance 10 --safety-margin-mm 0
```

Trim a solid border and keep a 2 mm safety margin:

```
python system_core\main.py trim-border --input input --output output\trimmed --to png
--tolerance 10 --safety-margin-mm 2
```

Rasterize PDF pages to PNG at 300 DPI:

```
python system_core\main.py pdf-rasterize --input input\sample.pdf --output
output\pdf_pages --to png --dpi 300
```

Export PDF to PNG by taking the largest embedded raster from each page:

```
python system_core\main.py pdf-export --input input\sample.pdf --output
output\pdf_export --to png --mode embedded
```

Export PDF to PNG as a full page render with text and vector content:

```
python system_core\main.py pdf-export --input input\sample.pdf --output
output\pdf_export_render --to png --mode render --dpi 300
```

Export PDF to PNG as a full page render with a custom DPI such as 200:

```
python system_core\main.py pdf-export --input input\sample.pdf --output
output\pdf_export_render_200 --to png --mode render --dpi 200
```

Extract embedded images from PDF without page rasterization:

```
python system_core\main.py pdf-extract-embedded --input input\sample.pdf --output
output\pdf_embedded_extract
```

Build a PDF from images in lossless-preferred mode:

```
python system_core\main.py images-to-pdf --input input --output
output\images_to_pdf.pdf --mode lossless
```

Notes

- JPG output is not mathematically lossless. Use PNG when lossless output matters.
- Plotter DPI-only mode keeps the original pixel dimensions and only rewrites output DPI metadata.
- `photo-sheet-width` does not resample source pixels by default: sheet width and DPI describe the final output sheet. If the selected DPI is too low for unchanged source pixels at the selected sheet width, the engine raises the sheet DPI automatically and writes a warning to the report.
- `trim-border` and `smart-crop-white` crop exactly to the detected meaningful pixels when `--safety-margin-mm 0`; larger values keep source background around the result using image DPI.
- The GUI can build a PDF after crop without resampling pixels: `PNG embedded` for lossless-preferred assembly or `JPG(75) embedded` for compact delivery.
- `pdf-export --mode embedded` takes the largest embedded raster from each page and is useful for single-page scanned PDFs that are really one big JPG inside a PDF wrapper.
- `pdf-export --mode render` rasterizes the whole page and keeps visible text/vector content in the PNG/JPG result.
- `pdf-export --mode render` accepts any positive DPI up to `1200`.
- PDF rasterization always renders the whole page appearance; embedded image extraction only pulls internal raster objects from the PDF.
- PDF normalization is page-based: the tool rasterizes PDF pages to TIFF, normalizes them, and assembles a new PDF.
- ICC conversion and ICC embedding are separate policies: color transform changes pixels, embedding only attaches profile metadata.
- Default policy: embed `sRGB` on save, do not embed `CMYK` on save, preserve image ICC in PDF workflows when useful.
- Multi-page PDF and TIFF page export uses numbered filenames with the `001_name` ... `999_name` pattern.

- Single-page PDF and TIFF export does not add a page-number prefix.
- The current project ICC set is intentionally small: `sRGB2014.icc`, `Photoshop5DefaultCMYK.icc`, and `CoatedFOGRA39.icc`.
- Folder processing preserves the source folder structure inside the selected output folder.
- The launcher does not ask for input/output paths on every run; it uses the project `input\` and `output\` folders by default and prints them before running a command.

Quick guidance

- Choose `sRGB` when the result is meant for screens, websites, or general RGB delivery.
- Choose `CMYK` when the result is meant for a print-oriented workflow.
- Choose Pillow sRGB when you want the built-in CMS path and do not need an external ICC file.
- Choose the color.org sRGB profile when you want an explicit external ICC-based sRGB target.
- Choose `Photoshop5DefaultCMYK.icc` for a broad classic CMYK workflow.
- Choose `CoatedFOGRA39.icc` when you specifically want that coated-print profile rather than a broader default CMYK path.

Operator Workflow

Select the operation first, then add source files through the Workbench and choose a separate target. Review format, quality, dimensions, color profile, metadata, overwrite policy, and multi-threading/RAM limits before starting. The GUI is an adapter over the same processing services used by CLI.

Conversion, crop, resize, color, PDF, photo-sheet, label-sheet, contact-sheet, diagnostics, and maintenance pages have different output contracts. Do not reuse a target folder from an unrelated operation without checking filenames and overwrite behavior.

Format Handling

Stable raster formats use the primary image pipeline. Extended formats may require optional codecs, external tools, or format-specific adapters. Best-effort inputs must report skipped frames, pages, layers, profiles, or metadata instead of silently flattening unexpected content.

PDF processing is document processing, not merely raster conversion. Verify page count, page size, orientation, transparency, embedded profiles, and print intent. Keep an untouched source when normalizing or rebuilding PDFs.

Color Management

Transform converts pixel values into the selected destination color space. **Embed** attaches or replaces profile metadata without necessarily converting pixels. These operations are not interchangeable.

For sRGB delivery, confirm whether the input already has a valid profile and whether web/office compatibility or colorimetric preservation is the priority. For CMYK, select the actual print condition and review dark tones, gradients, neutral grays, and out-of-gamut colors.

Never assume that a file named CMYK is ready for a specific press. Keep the chosen ICC profile with the job record and inspect the output in a color-managed application.

Quality And Acceptance

Open representative outputs at 100% and fit-to-page. Check dimensions, orientation, alpha, text, thin lines, sharpening, compression artifacts, color, metadata, and expected file count. For sheets, verify margins, spacing, labels, pagination, and print size.

Compare source and output counts and review every warning in the report. A successful exit code does not prove visual acceptance. Preserve sources and reports until delivery.

Resource Control

Multi-threading improves throughput only when memory, disk, and codec behavior allow it. Use conservative worker and RAM limits for large images, many-page PDFs, or high-resolution contact sheets. If the system starts paging, reduce concurrency rather than repeatedly restarting the same batch.

Safe Cleanup

Cleanup may remove temporary conversions, caches, logs, reports, and rebuildable runtime artifacts according to policy. It must preserve input, accepted output, config, ICC profiles, source code, canonical docs, and licenses.

Manifest As A Documentation Source

The GUI manifest is the structured catalogue of operations, formats, controls, defaults, tooltips, and backend actions. Human documentation should preserve the intent behind that structure: when to use each transform, what happens to color and metadata, which formats are best-effort, and how to judge the result. New manifest entries therefore require a matching explanation in the README or user guide, not just a visible button.

During release review, compare the manifest, GUI labels, command preview, output layout, and reports. This catches stale screenshots and prose while keeping the manifest itself machine-readable.